

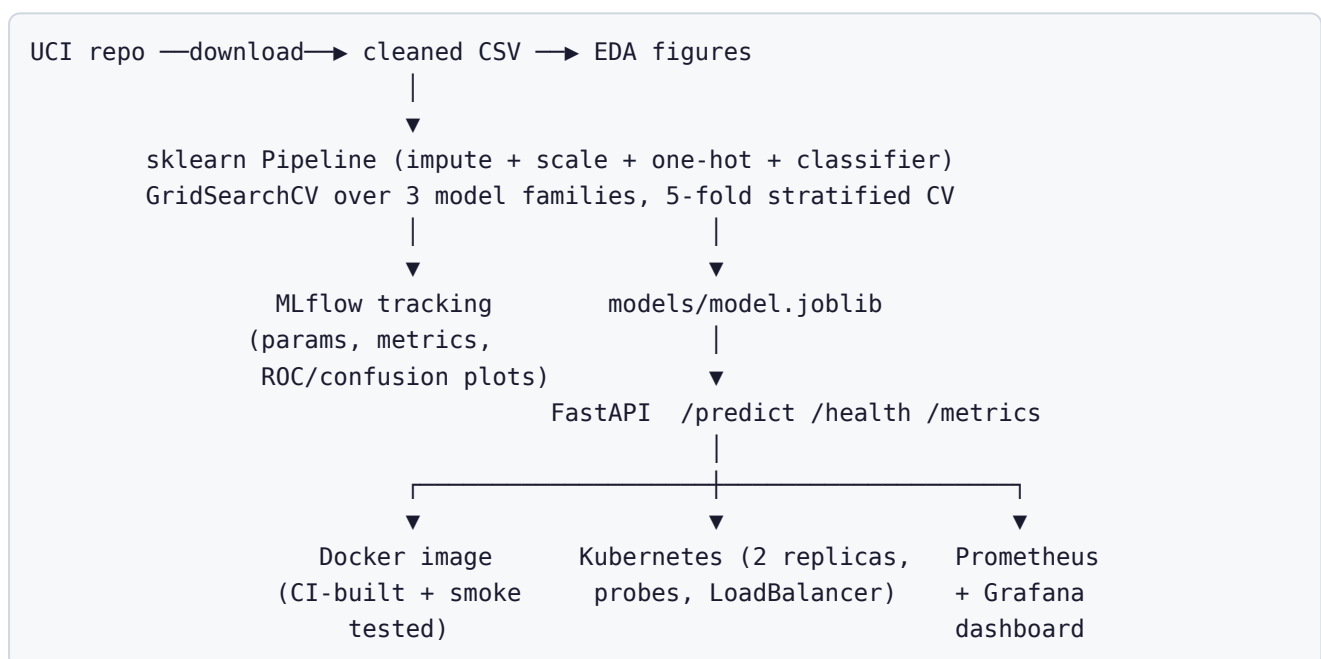
Heart Disease Risk Prediction — MLOps Project Report

Author: Diya Roy **Date:** July 2026 **Repository:** <https://github.com/diyaRoy46/heart-disease-mlops>

1. Project overview

This project builds and productionizes a binary classifier that predicts the risk of heart disease from routine patient measurements, using the UCI Heart Disease dataset (Cleveland subset, 303 patients, 13 features). The emphasis is on the *system around the model*: reproducible data acquisition, tracked experiments, automated quality gates, containerized serving, declarative deployment and live monitoring.

Pipeline at a glance



2. Data acquisition & preparation

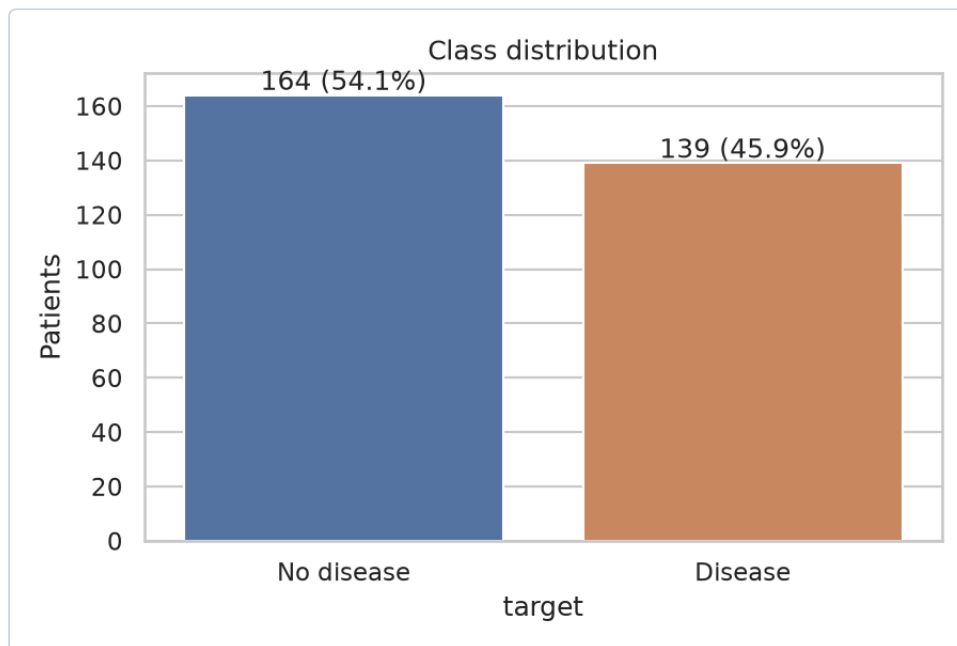
- `python -m scripts.download_data` fetches `processed.cleveland.data` from the UCI repository, names the 14 columns, parses `?` as missing, binarizes the target (`num > 0` → 1) and writes `data/heart.csv` (committed for reproducibility; the raw file is re-downloadable at any time).
- **Missing values:** only `ca` (4) and `thal` (2). They are *not* filled during cleaning — imputation happens inside the model pipeline so it is fitted on training folds only and replayed identically at inference time.
- **Encoding/scaling:** `ColumnTransformer` with median imputation + standard scaling for the 5 numeric features, and mode imputation + one-hot encoding

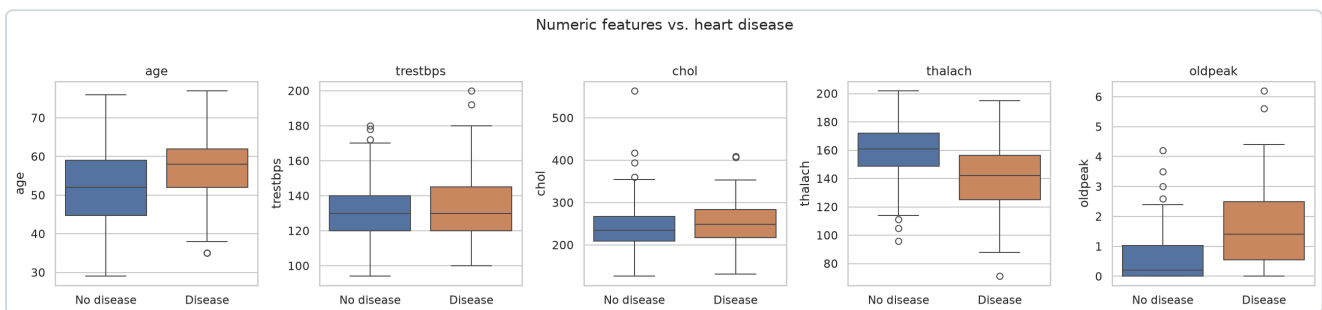
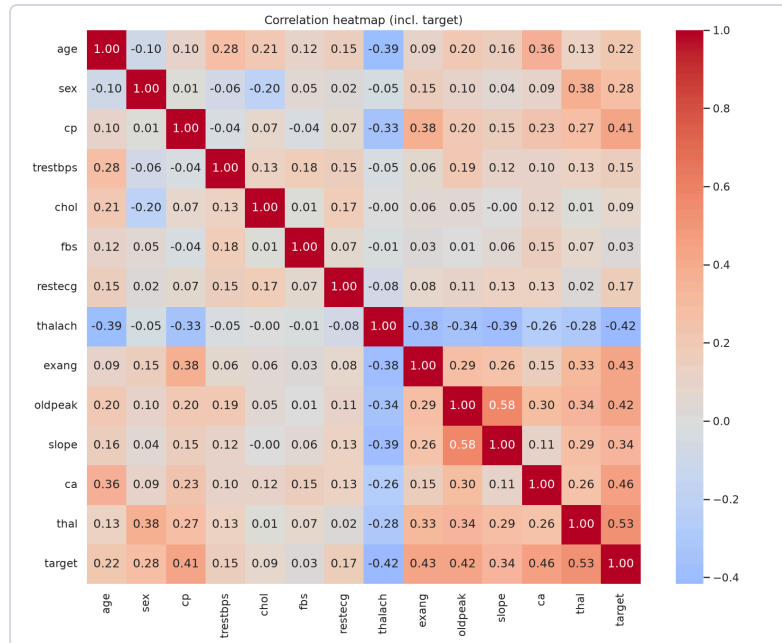
(`handle_unknown="ignore"`) for the 8 categorical features.

3. EDA findings

Figures in `reports/figures/` , summary in `reports/eda_summary.md` .

- **Class balance:** 164 without disease (54%) vs 139 with disease (46%) — near-balanced, so accuracy is meaningful but ROC-AUC is used for model selection anyway.
- **Strongest univariate signals** ($|\text{correlation}|$ with target): `thal` (0.53), `ca` (0.46), `exang` (0.43), `oldpeak` (0.43), `thalach` (-0.42), `cp` (0.41). Clinically plausible: exercise-related measurements dominate.
- Patients with disease show **lower maximum heart rate** (`thalach`) and **higher exercise-induced ST depression** (`oldpeak`); asymptomatic chest pain (`cp = 4`) is heavily disease-associated.
- `chol` and `fb`s correlate weakly with the target (0.09 / 0.03) but are kept: tree models can still exploit interactions, and the cost is negligible.
- No degenerate columns; one exact duplicate policy applied (dedup in cleaning).





Further figures in `reports/figures/` : `02_missing_values.png` , `03_histograms.png` , `06_categorical_by_target.png` .

4. Model development & comparison

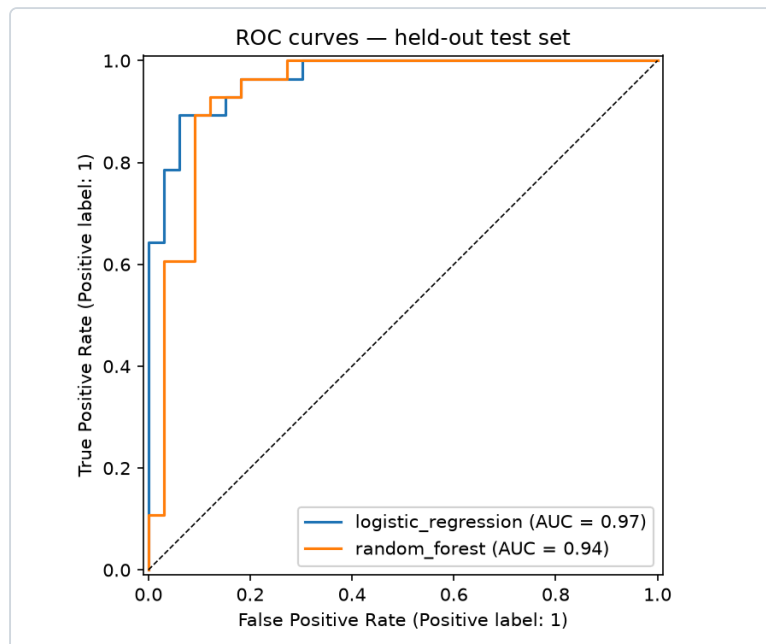
Three model families were tuned with `GridSearchCV` (5-fold stratified CV, ROC-AUC objective) on an 80/20 stratified split (seed 42), each wrapped in the same preprocessing pipeline:

Model	Grid	Best params
Logistic Regression	$C \in \{0.01, 0.1, 1, 10\} \times \text{class_weight} \in \{\text{None}, \text{balanced}\}$	$C=0.1$, balanced
Random Forest	$\text{trees} \{200, 400\} \times \text{depth} \{4, 8, \emptyset\} \times \text{min_leaf} \{1, 3\}$	200 trees, depth 4, leaf 3
Gradient Boosting	$\text{trees} \{100, 200\} \times \text{lr} \{0.05, 0.1\} \times \text{depth} \{2, 3\}$	(see MLflow run)

Held-out test results (61 patients):

Model	Accuracy	Precision	Recall	F1	ROC-AUC	CV ROC-AUC
Logistic Regression	0.885	0.839	0.929	0.881	0.965	0.903 ± 0.017
Random Forest	0.852	0.806	0.893	0.847	0.951	0.902 ± 0.030
Gradient Boosting	0.902	0.867	0.929	0.897	0.961	0.874 ± 0.025

Selection: Logistic Regression was exported (best test ROC-AUC, best CV stability, and the simplest/most interpretable model — a sensible property for a clinical screening aid). With ~300 rows, heavily regularized linear models are hard to beat. Recall of 0.93 means few missed disease cases; the `class_weight=balanced` setting deliberately trades a little precision for recall, which is the right direction for screening.



5. Experiment tracking (MLflow)

- Tracking store: `sqlite:///mlflow.db` (MLflow 3.x deprecates the `./mlruns` file store), override with `MLFLOW_TRACKING_URI`.
- Each model family = one run under experiment `heart-disease-classification`, logging: best hyperparameters, CV mean/std, all five test metrics, per-run **confusion matrix** and **ROC curve** PNGs, and the fitted pipeline as an MLflow sklearn model with input example.
- Browse with `mlflow ui --backend-store-uri sqlite:///mlflow.db`.

mf

flow

3.14.0

GenAI

Model training

heart-disease-cla...

Overview

Training runs

Observability

Traces

Sessions

Evaluation

Judges

Review

Datasets

Evaluation runs

Prompts & versions

Playground

Prompts

Agent versions

AI Gateway

New

Assistant

Beta

Docs

Settings

Experiments > heart-disease-classification >

Training runs

metrics.rmse < 1 and params.model = "tree"

All time

State: Active

Datasets

Sort: Created

+ New run

Columns

Group by

☐	Run Name	Created	Dataset	Duration	Source	Models
☐	gradient_boosting	30 minutes ago	-	2.2s	train.py	model
☐	random_forest	30 minutes ago	-	3.2s	train.py	model
☐	logistic_regression	30 minutes ago	-	3.6s	train.py	model
☐	gradient_boosting	34 minutes ago	-	2.2s	train.py	model
☐	random_forest	34 minutes ago	-	3.2s	train.py	model
☐	logistic_regression	34 minutes ago	-	3.7s	train.py	model
☐	logistic_regression	35 minutes ago	-	3.8s	train.py	model
☐	logistic_regression	35 minutes ago	-	8ms	train.py	-

Show more columns (20 total)

8 matching runs

mf

flow

3.14.0

GenAI

Model training

heart-disease-cla...

Overview

Training runs

Observability

Traces

Sessions

Evaluation

Judges

Review

Datasets

Evaluation runs

Prompts & versions

Playground

Prompts

Agent versions

AI Gateway

New

Assistant

Beta

Docs

Settings

heart-disease-classification > Runs >

logistic_regression

Overview

Model metrics

System metrics

Traces

Artifacts

Description

No description

Metrics (7)

Search metrics

Metric	Value	Models
test_accuracy	0.8852459016393442	model
test_precision	0.8387096774193549	model
test_recall	0.9285714285714286	model
test_f1	0.881355932203898	model
test_roc_auc	0.9653679653679654	model
cv_roc_auc_mean	0.9027904462687071	model
cv_roc_auc_std	0.016869568101549073	model

Parameters (5)

Search parameters

Parameter	Value
model	LogisticRegression
classifier__C	0.1
classifier__class_weight	balanced
cv_folds	5
n_train_rows	242

About this run

Created at

07/11/2026, 10:33:48 PM

Created by

dyaroy

Experiment ID

1

Status

Finished

Run ID

6e852303b7664724b9e79f5a6e0b88aa

Duration

3.6s

Source

train.py master 1148f0b

Registered prompts

-

Datasets

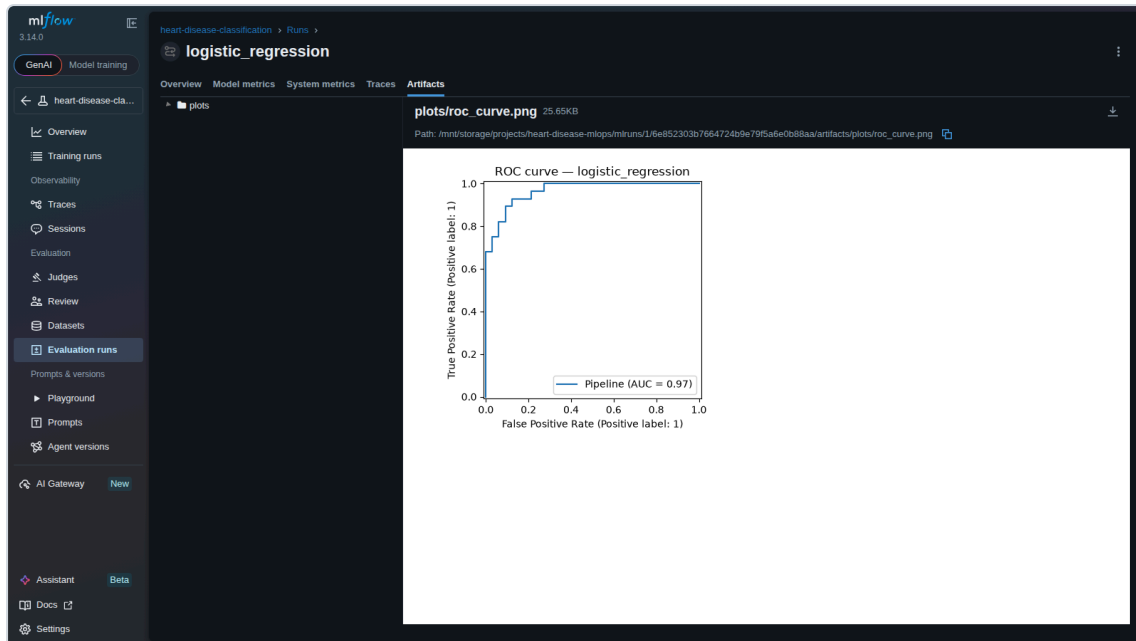
None

Tags

model_family: logistic_regression

Registered models

None



6. Model packaging & reproducibility

- Export format: single `joblib` file containing the **entire pipeline** (imputers, scaler, encoder, classifier) plus `models/metadata.json` (model name, params, metrics, MLflow run id, library versions, timestamp).
- `requirements.txt` (training) and `requirements-serve.txt` (serving, with model-critical packages pinned exactly) recreate both environments cleanly.
- Seeds fixed in `src/config.py`; retraining from the committed CSV reproduces the comparison table.

7. Serving API (FastAPI)

- `POST /predict` — validated JSON in (pydantic ranges from the UCI codebook; `ca` / `thal` optional), prediction + probability + model version out. Out-of-range input → HTTP 422; model failure → HTTP 500 with logged traceback.
- `GET /health` — used by Docker HEALTHCHECK and k8s probes.
- `GET /metrics` — Prometheus exposition (request counts/latency histograms via `prometheus-fastapi-instrumentator`, plus custom `model_predictions_total` counter labelled by predicted class).
- Every request is logged: method, path, status, latency, client.
- Interactive docs at `/docs` (Swagger UI).

8. Testing & CI/CD

Tests (pytest, 16 cases): data cleaning (target binarization, `?` parsing, dedup, schema of the committed CSV), preprocessing (no NaN survives, unseen categories don't crash), training (metrics bounded, quick end-to-end run logs to a temporary MLflow store, ROC-AUC > 0.8), API (health, valid/invalid predict, optional fields, metrics endpoint).

GitHub Actions (`.github/workflows/ci.yml`):

Job	Gate
lint	ruff — fails the pipeline on style/bug-pattern violations
test	pytest + coverage — fails on any test failure
train	reduced-grid training; uploads model, comparison and MLflow DB as artifacts
docker	builds the image, boots the container, polls /health , smoke-tests /predict

train and docker only run after lint and test succeed, so a red check blocks the artifact and image stages — the pipeline stops and reports clearly.

```

diyaroy@dev:~/heart-disease-mlops$ gh run list --limit 2
completed success Add deployment and monitoring evidence, final PDF report CI master push 29162223286 2m53s
completed success Add README, project report and EDA notebook CI master push 29161444781 3m3s
diyaroy@dev:~/heart-disease-mlops$ gh run view 29162223286
✓ master CI · 29162223286
Triggered via push

JOBS
✓ Lint (ruff) in 1m3s
✓ Unit tests (pytest) in 1m25s
✓ Docker build + smoke test in 37s
✓ Train model (smoke) in 1m21s

ARTIFACTS
model-artifacts

View this run on GitHub: https://github.com/diyaRoy46/heart-disease-mlops/actions/runs/29162223286

```

9. Deployment

Docker (verified locally): `docker build -t heart-disease-api .` → `docker run -p 8000:8000 heart-disease-api` → `/health` and `/predict` return correct responses. Image: `python:3.14-slim`, non-root user, HEALTHCHECK, serving-only dependencies.

Kubernetes (verified on Minikube v1.35 / Kubernetes v1.35.1): Deployment (2 replicas, readiness/liveness probes on `/health`, CPU/memory requests+limits, Prometheus scrape annotations) and a LoadBalancer Service (port 80 → 8000):

```

minikube image load heart-disease-api:latest # or build via minikube docker-env
kubectl apply -f k8s/
minikube service heart-disease-api --url # or: minikube tunnel

```

Both replicas reached `Running/Ready` and the API served predictions through the cluster service:

```
diyaroy@dev:~/heart-disease-mlops$ kubectl get all
```

NAME	READY	STATUS	RESTARTS	AGE
pod/heart-disease-api-854775c74c-mhzql	1/1	Running	0	49s
pod/heart-disease-api-854775c74c-xlvtf	1/1	Running	0	49s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/heart-disease-api	LoadBalancer	10.109.132.84	<pending>	80:31504/TCP	49s
service/kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	2m49s

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/heart-disease-api	2/2	2	2	49s

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/heart-disease-api-854775c74c	2	2	2	49s

```
diyaroy@dev:~/heart-disease-mlops$ minikube service heart-disease-api --url
http://192.168.49.2:31504
diyaroy@dev:~/heart-disease-mlops$ curl http://192.168.49.2:31504/health
{"status":"ok","model_loaded":true,"model_name":"logistic_regression"}
diyaroy@dev:~/heart-disease-mlops$ curl -X POST http://192.168.49.2:31504/predict -H 'Content-Type: application/json' \
-d
'{"age":57,"sex":1,"cp":4,"trestbps":140,"chol":241,"fbs":0,"restecg":1,"thalach":123,"exang":1,"oldpeak":0.2,"slope":2,"ca":
{"prediction":1,"label":"Heart disease","probability":0.841,"model_name":"logistic_regression","trained_at":"2026-07-
11T17:03:58.030060+00:00"}
```

10. Monitoring & logging

`docker compose up -d --build` starts API + Prometheus + Grafana:

- Prometheus scrapes `/metrics` every 5 s (target health verified `up`).
- Grafana auto-provisions the **Heart Disease API** dashboard: request rate by endpoint, p50/p95 `/predict` latency, predictions per outcome class, and 4xx/5xx error rates.
- The prediction-outcome panel doubles as a crude **drift signal**: a sustained shift in the predicted-positive rate flags a change in incoming data.
- Structured request logs (`docker logs heart-disease-api`) cover every call.



Prometheus

Query Alerts Status > Target health

Select scrape pool Filter by target health Filter by endpoint or labels

Target	Endpoint	Labels	Last scrape	State
heart-disease-api	http://api:8000/metrics	instance="api:8000" job="heart-disease-api"	2.108s ago 2ms	UP
prometheus	http://localhost:9090/metrics	instance="localhost:9090" job="prometheus"	4.477s ago 2ms	UP

11. Setup instructions (clean machine)

```
git clone <repo-url> && cd heart-disease-mlops
python -m venv .venv && source .venv/bin/activate
pip install -r requirements-dev.txt
python -m scripts.download_data # data
python -m src.eda # EDA figures
python -m src.train # tracked training + model export
pytest && ruff check src tests scripts
uvicorn src.api.main:app --port 8000
# or the full stack:
docker compose up -d --build
```